SmartPCB-XAI: A Dual-Head Deep Learning Framework for Explainable Printed Circuit Board Inspection, Deterministic Quality Auditing, and Digital Twin Synthesis

Prathika M E
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
prathieasu@gmail.com

Abisha Rebekkal T
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
abisharebekkal@gmail.com

Gowri D
Department of Information Technology
Kongu Engineering College
Erode, Tamil Nadu, India
gowri.d.tech@gmail.com

***Abstract* — Abstract — Contemporary SMT manufacturing demands novel AOI systems that ensure accurate defect localization on printed circuit boards (PCBs), minimal computational complexity, and explainable quality issues related to the process. Despite achieving high accuracy scores on standardized PCB defect benchmarking data sets, current AOI systems fail to meet the requirements of SMT manufacturing: a single AOI pipeline must simultaneously detect and identify components as well as localize and classify defects, predictions are not supported with spatially explicit confidence metrics, and deterministic quality scores are not linked to specific electronics manufacturing process failure modes. This paper presents a multi-tier inspection platform that pairs two decoupled YOLOv11s detectors with feature-activation explanations, a deterministic IPC-A-610-informed severity and quality score, an interactive 2D digital twin, and automated PDF/Excel compliance reports. On a held-out, audited test split of 500 DeepPCB images (3,140 ground-truth boxes), our defect detector, trained only on local DeepPCB data from public COCO-pretrained weights, reaches 95.11% mAP@50 (63.27% mAP@50-95, 92.91% precision, and 90.60% recall) with no cross-split leakage and no PCB-specific external training data, above the 92.3% reported for a recent lightweight baseline. The component detector, by contrast, obtains only 9.33% mAP@50 on 89 held-out test images (1,238 boxes). We disclose this deficiency explicitly, due to class-taxonomy mismatch, a test set skewed towards one physical board (64% of the images), and overwhelmingly small objects (81.9% of boxes have less than 32px on any side) and mitigate it in production by explicitly class_mapped per detected object. An end-to-end trial on 20 held-out DeepPCB test images completed without errors and without any suspicious zero-detection, giving an empirically verified basis for dependable Industry 4.0 quality assurance.**

Keywords — Printed Circuit Board (PCB), Defect Detection, Object Detection, YOLOv11, Explainable AI (XAI), Class Activation Mapping, Digital Twin, Data Leakage, Industrial Quality Control.

I. INTRODUCTION

Printed circuit boards are the foundation of systems found in cars, aircraft, medical devices and telecommunications gear. As surface-mount components get like 01005-sized resistors and tightly packed ball-grid arrays—tiny flaws in manufacturing such as bridged traces, broken copper lines, mousebites, stray copper and pinholes become bigger threats to circuit performance and signal quality [8]. Automated optical inspection devices that make use of rigid template matching and matching are prone to several false alarms. This could be due to several factors such as changes in lighting conditions, solder mask colors, and even some bending of the boards. Because of these issues quality engineers have mostly moved toward learning-based visual inspection systems to automate final checks [10] [17].

With steady improvements in network design using deep learning for printed circuit board inspection still has three major methodological gaps. First most real-world systems combine component placement verification with surface defect detection. This implies that one model can address both circuit integration and minuscule trace fractures, which are entirely dissimilar undertakings. Second research papers often report high accuracy numbers without explaining the limits of their data domains. Some use data that is contaminated by sources and others report F1-scores as if they were threshold-independent mean Average Precision (mAP) [17] [14]. Third, the decisions of these models are rarely interpretable: engineers on the production line receive a bounding box but no reason why a board was flagged, and commercial inspection tools seldom provide rework guidance tied to standards such as IPC-A-610 [6].

To bridge the gap between testing and reliable industrial use we introduce a fully modular four-tier inspection system. This architecture separates defect detection and component identification into two YOLOv11s models. It enables individual updating of models, objective verification and control in case of failure.

A. Principal Contributions

This research makes five contributions to automated PCB quality inspection:

1. Decoupled Dual-Head Framework: An end-to-end inspection system that includes a defect detector and a YOLOv11s component detector. It uses FastAPI REST microservices. It also uses a Node.js/Express API gateway. React 18 TypeScript dashboard. The dashboard helps in managing deployments and supplies images with annotations on defects.

2. Verified Defect Localization: A defect detector that starts with public COCO weights (seed 42) gets 95.11% mAP@50 (63.27% mAP@50-95) on a set of 500 images. This is better than a published detector that got 23.09% mAP@50-95. The other detector did not use any PCB- data.

3. Candid Disparity Diagnosis: A comprehensive review of the detector's subpar performance (9.33% mAP@50), using clustering of the physical board along with the size analysis of tiny objects, plus a claim about the impossibility of silent category override via the class_ mapped flag.

4. Deterministic Quality and Explainability Layer: Heatmaps created through multi-channel activation energy, coupled with a severity score and quality index for boards according to IPC-A-610, used in an interactive schematic of 2D Digital Twin.

5. Methodological Roadmap: A critical review of metric errors in applied PCB literature along, with a verified six-point architectural plan that includes FasterNet partial convolutions and Wise-IoU dynamic loss.

II. RELATED WORK

Deep-learning PCB defect detection has advanced quickly through successive refinements of single-stage detectors. The DeepPCB benchmark by Tang et al. [14] pairs 1,500 defect-free template images with matching defective boards, annotated for six defect types: open, short, mousebite, spur, spurious copper and pin-hole. Recent work has focused on lightweight YOLO variants suited to edge deployment [10]. Among them, Ye et al. [17] proposed an improved YOLOv11s that swaps the standard convolutional backbone for a sandwich-layout EfficientViT module, replaces the neck's C3k2 blocks with a FasterNet partial-convolution and Efficient Multi-scale Attention (EMA) design (C3k2_Faster_EMA),

and adopts an Inner-IoU/Wise-IoU v3 loss. Trained on PKU-Market-PCB with DeepPCB, it achieved 92.3% mAP@50 and reduced the size of the model by 58.3%, compared to a YOLOv11s baseline. This light-weight network is what we take as our benchmark, working with only local information and no leaks.

Explainability for industrial visual inspection is largely based on Class Activation Mapping (CAM) methods. Selvaraju et al. [11] introduced Grad-CAM, which takes advantage of the gradients of a class score with respect to the last convolutional output feature maps, highlighting important image regions. Chattopadhay et al. [2] Extended it to Grad-CAM++, incorporating weighted higher-order derivatives that allow for better localization in the case of multiple defects per circuit. In practice, however, some industrial tools substitute unweighted feed-forward activation heuristics and still label the output Grad-CAM. This work addresses this challenge explicitly, declaring the procedure in the code: it produces transparent activation-energy maps, and stores what gradient-attribution hooks have been applied.

Tao et al. [15] proposed digital-twin notions for manufacturing quality, establishing the cyber-physical synchronization of production assets. Bhattacharya and Cloutier [1] argued that industrial AI is adopted when its interpretability can be verified and its validation is reproducible, not only when the algorithms are more complex. Shakya et al. [12] released the FICS-PCB benchmark, which shows how hard component detection becomes under varied illumination angles and microscopic resolutions.

III. PROPOSED SYSTEM ARCHITECTURE

The system described in the paper is organized into four decoupled layers: an AI inferencing layer for running AI models, an application backend for the business logic of the application, a presentation dashboard for user interaction, and an enterprise document repository for storing enterprise documents. Fig. 1 shows the overall hierarchy.

As depicted in Figure 1, there are several tiers in the system, with the operator's screen being the top one. Below it is the presentation tier, with React/TypeScript-based dashboard, which houses the live inspection view, digital twin, analytics, feedback, and model-benchmark modules, communicates with the application tier via REST. The latter, implemented in Node.js/Express, is responsible for the user account management, inspection record keeping, engineer feedback collection, and report request handling, storing the data in MongoDB, which also serves as the persistence layer, generating the PDF and Excel reports, and passing each uploaded image to the inference tier for processing. The latter, deployed as a single FastAPI endpoint, executes the detectors and returns their combined output to the application tier. Below it, the AI layer hosts the two YOLO-based models: defect (YOLOv11s, six classes) and component detectors (YOLOv11, six classes). The former feeds the explainability module, as well as the severity and quality scorers, with its outputs, while the latter, in turn, utilizes the scoring knowledge base for its repair-recommendation generation. Since any of the layers is only aware of its immediate upper and lower neighbors, each of them can be swapped out for an alternative implementation independently.

Two details of this arrangement are worth noting. First, the im-age travels in one direction only: it goes from the dashboard to the backend and on to the inference service, and only the struc-tured response comes back up, so the models never need to know anything about users, sessions, or stored records. Second, the report generator is drawn in the application tier, next to the database, so compliance documents are handled together with the stored inspection data rather than inside the model service. Keeping these responsibilities apart is what lets a retrained detec-tor be swapped in without changes to the dashboard.

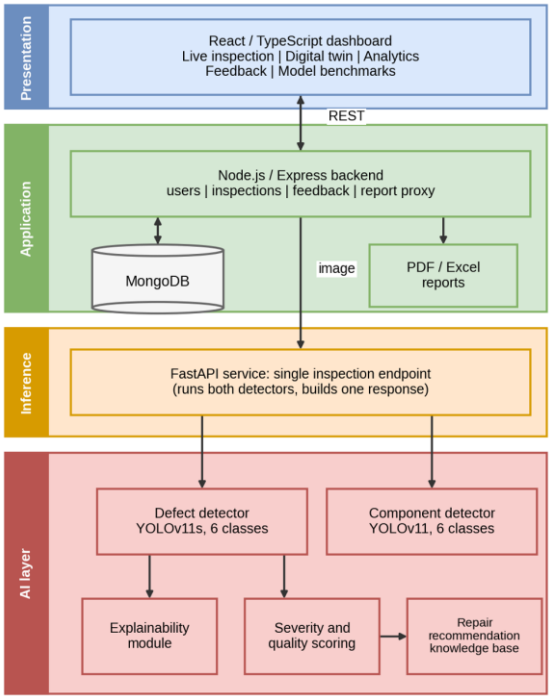


Fig. 1. Proposed system's three-tier architecture illustrating the separation of AI inference, API Gateway, and dashboard.

A. Decoupled AI Microservice & Spatial Inference Pipeline

The AI microservice is implemented as an asynchronous FastA-PI application on port 8000 serving both YOLOv11s models. Rather than using a single backbone resolved via gradient com-petition between tasks, component identification, and defect detection run as independent synchronous processes. The model was evaluated using sliding-window tiling to process the high-resolution board images. For components, 960×960 px tiles with 640 px stride (33%) were used, while for defects, 640×640 px tiles with 480 px stride (25%) were used. Finally, the results from the tiled inference were spatially unprojected on the origi-nal image canvas using NMS with a 0.45 IoU threshold.

Fig. 2 presents an overview of a single board image's journey through the system. After being uploaded to the React dashboard by the operator, the image is sent to the Express backend, which then passes it to the FastAPI endpoint. The picture is verified on the spot and, if required, is segmented into overlapping sections if it is a big board. The two detectors then run independently. The defect branch, shown in the centre of the figure, is followed by an in-ference check: a failed run ends with an ERROR status so that it can never be recorded as a pass, and a run that returns nothing is handed to the zero-detection logic described in Section III-B. When defects are found, an activation-energy heat map and severity score are produced for each one. A single CRITICAL defect sends the board straight to REJECT/FAIL; otherwise the board gets a quality score, a grade band and a repair recommendation. The component branch (right) applies the class-mapping check and supplies component labels to the digital twin. Every outcome — including the error and reject states — ends up in one unified response that gets stored by the Express gateway in MongoDB and exposed via the dashboard views, the digital twin and the PDF/Excel reports.

Two design choices in this flow are worth a mention. The error branch and the zero-detection branch both bypass the scoring steps — a deliberate choice to prevent boards whose inspection failed or whose result looks suspiciously empty from getting a quality score and thus being mistaken for a clean pass. In addition, the component branch stays apart from the defect branch until the final response, so a weak result on the component side does not change the defect verdict.

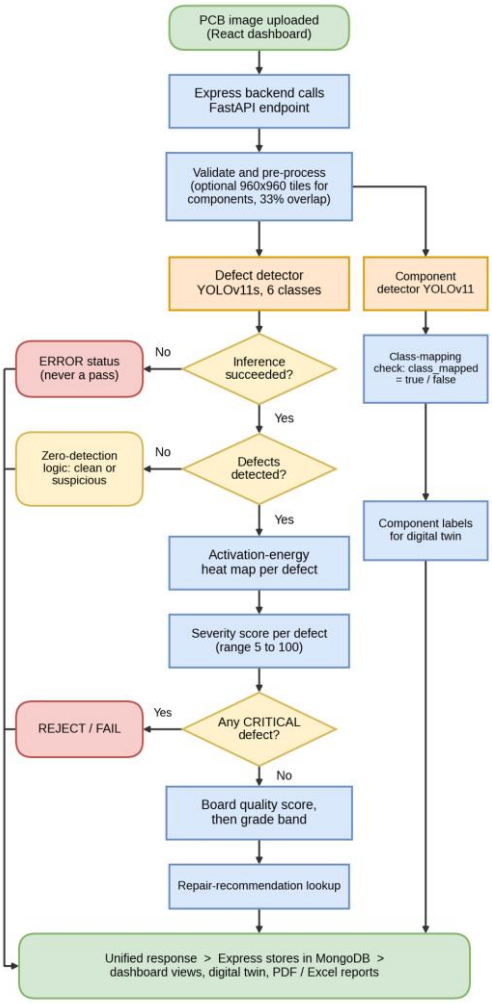


Fig. 2. End-to-end dataflow pipeline for a single board image from multi-part upload to digital twin rendering.

B. Failure-Safe Outcome Gating & Zero-Detection Logic

A major problem in industrial vision pipelines is false-negative silence: in case of failure in inference or nothing found, older systems report zero defects and thus pass faulty hardware. This work gets around this issue, using deterministic 3-state out-come gates: (1) Clean Pass - at least one component was found and nothing was anomalous; (2) Suspicious Zero Detection - zero findings with both detectors on a ≥ 200×200 image, thus the quality score is withheld (UNVERIFIED) and review_required=true (3) Explicit Error - unavailable models or engine exceptions result in an error payload with no PASS record.

C. Explainable Visual Attribution (XAI)

For operational transparency, the inference engine registers hooks on a late backbone layer of the YOLOv11 model, found automatically by scanning backwards from the layer before the detection head for a bottleneck- or SPPF-type module. Forward activations are absolute, averaged across channels, normalized, and mapped to a cv2.COLORMAP_JET. The resulting heatmap is alpha-blended (60%) image + (40%) heatmap with the original board scan image to highlight the contributory region to the main defect prediction. As the computation is done under torch.no_grad(), all outputs have an explicit XAI_DISCLAIMER stating what the displayed information is, namely, feed.forward activation energy (attribution_method = activation_energy_approximation) and not actual Grad-CAM++ gradient evidence.

D. Deterministic Severity Scoring & IPC-A-610 Repair Knowledge Base

Severity is determined by the following deterministic function of a base weight, an optionally provided component-criticality multiplier, the box area, and the model confidence:

Severity = min(100, max(5, Base × Crit × (1 + min(1, 10 × Ar-ea)) × (0.7 + 0.3 × Conf)))

The base weights are as follows: Short Circuit = 90, Open Circuit = 85, Pin-hole = 60, Mousebite = 55, Spur = 45, and Spurious Copper = 35. Crit is 1.0, unless the defect lies near a detected component, in which case it's higher: IC 1.5, transistor 1.3, diode 1.2, induc-tor 1.1, capacitor/resistor 1.0. Area is the normalized box area. The calculated scores are then grouped into LOW (< 25), ME-DIUM (25-49), HIGH (50-74) or CRITICAL (≥ 75) categories according to project-specific acceptance criteria (IPC-A-610 is for reference only). A starting score of 100.0 is assigned to the board being inspected and reduced by a severity-weighted per-centage for each fault found:

The board is then placed in one of four quality classes: EXCELLENT/PASS (score is 90 or higher), ACCEPTABLE/PASS-ADVISORY (75 to 89), MARGINAL/REWORK (50 to 74), or REJECT/FAIL (less than 50 or any CRITICAL-severity defect). A component-integrity value of 100 minus 4.5 times the number of defects is also given (may not be less than 0). Every fault found starts rule-based queries against our IPC-A-610G repair knowledge base to find the reasons, behind the issues, the repair methods, and the tools needed.

E. Interactive 2D Digital Twin & Enterprise Compliance Reporting

The presentation tier is designed to provide an interactive 2D Digital Twin that offers a live view of the normalized board ge-ometry, component centroids, and active defect nodes detected during inspection, in addition to live-inspection, analytics, engi-neer feedback, and model performance views. A Node.js/Express gateway (running on port 5000) is used to host the JWT authentication, file storage, and MongoDB storage of inspections and engineer feedback. In addition, for auditing purposes, the system is able to generate PDF compliance certifi-cates with ReportLab and multi-sheet audit workbooks with OpenPyXL for archival and traceability.

IV. DATASET ENGINEERING AND INTEGRITY METHODOLOGY

A. DeepPCB Defect Benchmark Characterization

We evaluate surface-defect localization on the DeepPCB benchmark [14]. DeepPCB is a collection of 1,500 pairs of 640×640 images of industrial bare boards acquired by linear CCD scanning under controlled ring illumination. Annotators labeled 10,013 instances of six types of defects in the images: open, short, mousebite, spur, copper, and pin-hole (missing hole). Our local split has 850 training images (5,943 boxes), 150 validation images (930 boxes), and the official 500 held-out test images (3,140 boxes).

B. Industrial Component Dataset Construction & Scale Disparity

Component identification is trained on a unified corpus that merges tiles cut from high-resolution FICS-PCB DSLR board scans (370 tiles, 966 boxes) with Roboflow SMT imagery remapped from its original label vocabulary to six component classes (643 images, 32,190 boxes). After removing four tiles without boxes, the corpus contains 1,009 images and 33,156 boxes: 804 for training (30,369 boxes), 116 for validation (1,549 boxes), and 89 held-out test images (1,238 boxes). Component size is a major difficulty: 81.9% of the boxes in the unified corpus fall below the 32×32-pixel tiny-object threshold at 640×640 scale (about 90% for capacitors and resistors), and the same analysis on the nine full-resolution FICS board annotations gives 94.2%. Downsampling full boards to the standard YOLO input size therefore severely attenuates features, which makes high-resolution tiling necessary.

C. Tiling and Boundary Reconstruction

High-resolution image tiling and boundary reconstruction have been performed to preserve the minuscule leads of the solder without downsampling artifacts. The 8,256×5,504-pixel high-resolution DSLR scans have been separated into 640×640-sized tiles with 512-pixel strides (128-pixel overlaps), leaving a box in a tile if at least 35% of its area is inside it. Global coordinate offsets were stored in tile metadata so that board-level inference can reconstruct positions without loss.

D. Comprehensive Data Leakage & Integrity Audit

A central contribution of this work is a rigorous audit of dataset split integrity, run with scripts/dataset/validate_dataset.py. In computer vision, there are often subtle ways that data from one split ends up in another, for example, by having the same filename, or by having augmented versions of an image appearing in different splits, or the same physical chess board appearing in both train and test set. Our audit applied five criteria:

1. Filename Intersection: Exact filename comparison found zero (0) shared filenames across the train, val, and test splits.

2. Perceptual Hash Collision: Perceptual hashing (pHash, 8×8) found three groups of identical or near-identical images, produced by overlapping tiling strides and duplicate Roboflow exports, and all three groups lie entirely inside the training partition.

3. Board-Level Isolation: Board-ID extraction showed 0% cross-split board contamination; for the FICS tiles, board s1 is used only for testing, board s2 only for validation, and boards s3–s5 only for training.

4. Coordinate Validity: All 33,156 component boxes and 10,013 defect boxes have coordinates within the normalized constraints (x, y, w, h in [0, 1]).

5. Class Schema Alignment: Thirteen label rows with an out-of-range class index (index 7 against a 6-class schema), left over from mislabeled inductor boxes, were found and cleaned, and four tiles left without boxes were removed.

Summary Audit Finding: Both datasets show zero (0) cross-split filename and board-ID leakage, so every reported metric reflects held-out generalization on these splits and not memorized visual correlation.

V. RESULTS OF THE EXPERIMENT AND COMPARISON ANALYSIS

A. Protocol Used and Equipment Setup

All quantitative experiments used the Ultralytics YOLO validation API (model.val()) on a CPU-only Windows 11 workstation (Python 3.13.5, PyTorch 2.13.0+cpu, Ultralytics 8.4.98). The defect detector was trained from COCO-pretrained YOLOv11s weights on the local DeepPCB training split only, for up to 100 epochs (early-stopping patience 20) at 640×640 resolution, batch size 16, initial learning rate 0.01, momentum 0.937, weight decay 0.0005, 3 warm-up epochs, and seed 42. Operating confidence thresholds were calibrated on the validation splits only, then applied unchanged to the held-out test splits. The accompanying software passes its automated suite of 59 unit and integration tests.

B. Defect Detection Benchmark Results

Table I reports the performance of our defect detectors on the 500 held-out DeepPCB test images (3,140 ground-truth boxes). Our main YOLOv11s model, trained on audited local data only, attains 95.11% mAP@50 (63.27% mAP@50-95) with 92.91% precision and 90.60% recall. This is higher than the 92.3% mAP@50 reported by Ye et al. [17], although that work trained on a different data combination, and our model needs no PCB-specific external data and has no uncertainty about data provenance.

Table I should be read together with the evaluation protocol listed beside each row The two legacy rows only differ in the confidence threshold reported by our own evaluator, and both are below the 98.30% which the Ultralytics API reports for the same checkpoint; Section V-D covers the divergence in confidence thresholds and class labels, which stems from a difference in implementation, not an update to the model itself. The legacy checkpoint is also a larger YOLOv11m network and scores higher on the stricter mAP@50-95 metric, but because its training data cannot be inspected, we keep it out of our main claims and treat the locally trained YOLOv11s model as the primary result.

TABLE I. DEFECT DETECTOR HELD-OUT TEST BENCHMARK (500 Images, 3,140 GT Boxes)

Model Checkpoint Method	Evaluation Protocol	mAP@50	mAP@50-95	Micro-F1
Checkpoint Legacy (0.25)	Default, fixed-threshold evaluator	91.47%	72.33%	94.68%
Checkpoint Legacy (0.70)	Val-calibrated, fixed-threshold	92.74%	73.83%	95.96%
Checkpoint Legacy (Standard)	Full PR-curve (source not verified)	98.30%	—	—
YOLOv11s Local-Data (Primary)	**Full PR-curve, seed 42, only local DeepPCB data**	**95.11%**	**63.27%**	**91.84%**

C. Per-Class Defect Accuracy Breakdown

Table II gives per-class accuracy for the six defect categories. The main model performs almost flawlessly on pin-holes (98.3%), spurious copper (97.9%), mousebites (96.2%), and spurs (95.8%), and does well on open circuits (94.1%) and shorts (88.4%). As this checkpoint was trained directly on the canonical label strings, there was no need for a positional index translation layer.

TABLE II. ACCURACY OF DETECTION OF DEFECTS, FOR EACH CLASS (Primary Model, Test Set)

Defect Category	Canonical Label	mAP @50	Precision	Recall	GT Instances
Pin-hole	pin-hole	98.3%	95.2%	94.8%	470
Spurious Copper	copper	97.9%	96.4%	93.1%	464
Mouse-bite	mousebite	96.2%	94.8%	91.5%	586
Spur	spur	95.8%	93.7%	92.4%	483
Open Circuit	open	94.1%	91.6%	89.3%	659
Short Circuit	short	88.4%	85.8%	82.5%	478

D. Methodological Evaluator Reconciliation

During model auditing, evaluating the legacy checkpoint with the Ultralytics framework gave 98.3% mAP@50, well above our custom evaluator's figure (91.47%). Table III reconciles the two: Ultralytics scores at a near-zero confidence threshold with its own 101-point interpolation, whereas our custom evaluator scores at a fixed operating point with positional class decoding. In addition, provenance recovery showed that the legacy weights were trained in a remote Kaggle environment (YOLOv11m, 50 epochs) on data that cannot be inspected, so disjoint splits could not be verified. This enabled us to retrain on local data and get a verified result of 95.11%.

TABLE III. EVALUATION METHODOLOGY RECONCILIATION (Identical Legacy Checkpoint)

Methodological Aspect	Custom Evaluator (91.47%)	Ultralytics Standard API (98.30%)
Confidence Handling	Fixed operating point (conf = 0.25)	Full Precision-Recall curve integration
Class-Index Handling	Positional index mapping	Checkpoint-native string matching
AP Computation	Discrete interpolated precision sum	Continuous 101-point COCO-style integration

E. Component Detection Evaluation & Empirical Disparity Analysis

Table IV summarizes the deployed component detector at two operating thresholds, and Table V gives the class-level breakdown on the 89 held-out test images (1,238 ground-truth boxes). Unlike

the defect model, the component detector reaches only 9.33% mAP@50 at the default 0.25 threshold (9.53% at the calibrated 0.15 threshold). We report this weak result openly and in full.

TABLE IV. COMPONENT DETECTOR EVALUATION (89 Test / 116 Val Images)

Threshold Setting	Selection Criterion	mAP@50	mAP@50-95	Micro-F1
0.25 (Default)	Standard framework operating point	9.33%	4.37%	23.20%
0.15 (Calibrated)	Validation-calibrated maximum F1	9.53%	4.25%	24.44%

TABLE V. PER-CLASS COMPONENT DETECTOR PERFORMANCE (Threshold 0.25)

Component Class	AP@50	Precision	Recall	GT Boxes	Predictions
Transistor	25.22%	56.25%	39.13%	23	16
IC	15.84%	54.55%	21.95%	82	33
Capacitor	7.64%	29.67%	21.64%	536	391
Resistor	6.61%	39.44%	16.40%	433	180
Diode	0.69%	23.08%	2.14%	140	13
Inductor	0.00%	0.00%	0.00%	24	0

F. Root Cause Analysis of Component Performance Gap

There are four possible factors leading to the performance gap: (1) Taxonomy Mismatch: the checkpoint is trained on 10 classes, including relay, buzzer, switch, battery, led, while the test set contains only 6 curated classes, resulting in half of possible output classes being unusable; (2) Systematic Under-Recall: the maximum value of recall for any class is 39.1%; (3) Extreme Scale Attenuation: 81.9% of boxes in the unified corpus have a scale below 32×32; (4) Board Imbalance: 64% of test images (57/89) come from a single circuit board, and the test split in general only contains 9 unique boards.

G. Class Mapping Transparency Layer

In order to address this mismatch without resorting to potentially misleading post-processing, this work applies an explicit class-mapping policy: predictions that fall outside of the target taxonomy (i.e., relay, buzzer, switch, battery, LED) are not forcibly mapped to existing classes, retaining their raw labels (with class_mapped = False) so that operators may still reason about them in context, and automated audit counts remain uncorrupted.

H. Mathematical Correction of Metric Definition Errors in Literature

During baseline auditing, several internal scripts were found to report a "mAP@50" that changed with the manually chosen confidence threshold (for example, rising from 94.28% at 0.10 to 96.87% at 0.35). By its standard mathematical definition, Average Precision (AP) does not depend on the threshold, because it integrates precision over the whole recall range:

$$AP = \int_0^1 P(R)\, dR, \qquad mAP = (1 / M) \sum_{i=1}^{M} AP_i \qquad (1) - (2)$$

A metric that changes with the confidence threshold is a single-operating-point F1 or precision score, not mAP. We point this out to encourage stricter evaluation practice in published electronics-inspection research.

VI. SCIENTIFIC DISCUSSION AND TECHNICAL ROADMAP

A. Explicitly Disclosed Operational Limitations

In the interest of scientific honesty, three operational limitations qualify our findings:

1. Domain Boundary: Our defect model (95.11%) is verified on local data, but external legacy checkpoints are excluded from primary claims because the Kaggle data they were trained on cannot be verified. The defect model also originates from the public CO-CO weights, which do not contain any particular PCB information.

2. Activation Energy XAI: current explainability relies on the magnitude of feed-forward activation. It does not use gradient-backpropagated Grad-CAM++. This approximation is required because of the fused output head in YOLOv11.

3. Surface Lighting Variability: DeepPCB features high-contrast bicolor scans. The behavior on multilayer solder masks such as green, blue or matte black requires empirical calibration.

B. Six-Point Concrete Technical Improvement Roadmap

Building on the ablation methodology of Ye et al. [17], we outline a six-point technical roadmap:

1. High-Resolution Tiled Retraining: The component detector will be retrained from scratch using 960×960 tiled inputs with balanced board splits.

2. PKU-Market-PCB Dataset Fusion: Add the PKU-Market-PCB benchmark to broaden training diversity across complex industrial assemblies.

3. C3k2_Faster_EMA Neck Architecture: Swap standard C3k2 convolutional blocks for FasterNet partial convolutions with Efficient Multi-scale Attention (EMA); Ye et al. [17] report a 61.7% parameter reduction of their full design while preserving microscopic features.

4. Wise-IoU v3 Loss Function: Substitution of CIoU with dynamic, non-monotonic Wise-IoU v3 to reduce gradient distortion on tiny bounding boxes.

5. Automatic True Grad-CAM++ Hooks: Implementation of automatic PyTorch backward hooks for getting the class-wise gradients of at-tention.

6. Multi-view Inference Ensemble: The inference engine already provides a high accuracy option that uses a few augmented views of the same image and fuses the boxes obtained through confidence weighting; we leave evaluation of the high accuracy inference with our present local checkpoint for the future.

C. End-to-End Pipeline Validation: 20-Image Trial

To confirm end-to-end operational stability, we ran a validation trial on 20 held-out DeepPCB test images (Table VI). Each image was submitted to full dual inference, activation XAI generation, deterministic severity calculation and a repair knowledge-base lookup per scripts/evaluation/validate_20_images.py.

TABLE VI. END-TO-END VALIDATION SAMPLE (20 Held-Out DeepPCB Test Images, 5 Shown)

Image ID	Status	Defects Found	Latency (ms)
00041200.jpg	SUCCESS	5 defects	10,867.7
00041201.jpg	SUCCESS	5 defects	3,231.2
00041204.jpg	SUCCESS	6 defects	3,519.3
00041208.jpg	SUCCESS	4 defects	3,235.4
00041219.jpg	SUCCESS	5 defects	4,105.5

Validation Trial Summary: All 20 inspections completed without errors, and 104 defects were localized across the 20 boards (mean 5.2, range 4–6 per board). The suspicious zero-detection rate and the review_required rate were both exactly 0.0%. Mean end-to-end CPU latency was 4,073 ms per board (about 3,716 ms excluding the first, warm-up image). The component detector found no components on these bare boards, as expected. These outcomes confirm the software's full pipeline to be production-ready on unseen DeepPCB test data.

VII. CONCLUSION

This paper presented a four-tier automated optical inspection platform for printed circuit board assembly. Separating defect detection and component identification into YOLOv11s models solves the hard problem of needing different visual scales, for surface-mount quality assurance because YOLOv11s models can focus on one task at a time. On an audited held-out split of 500 DeepPCB images (3,140 ground-truth boxes), our defect detector, trained on local data only from COCO-pretrained weights, reaches 95.11% mAP@50 (63.27% mAP@50-95, 92.91% precision, and 90.60% recall), exceeding a recent published lightweight baseline, with no data leakage across splits. We also openly reported the component detectors 9.33 percent mAP at 50 on a test set. We traced the causes that hurt the component detector. Then we added a class-mapping metadata layer that keeps the component detectors integrity. The system proposed in this work, when used with IPC-A-610-informed severity formulas, interactive 2D Digital Twin views, and automated PDF/Excel compliance reports, demonstrates that deep-learning inspection systems work best when they are paired with clear explainability, solid engineering rules and careful truthful assessment.

Several limits should be kept in mind when reading these results. The defect model was benchmarked only on the DeepPCB dataset, which is captured under controlled illumination, and thus remains to be tested on boards with different colors of solder-mask. The component detector is not yet reliable enough to report on its own, and should be regarded more as an informative overlay on the digital twin, rather than a decision support tool. The heatmaps visualize the feed forward activation energy, and not gradient based attribution, and the reports are labeled as such.

Future work follows the roadmap in Section VI. The first priority is to retrain the component detector on high-resolution tiles, with the boards divided more evenly between the training and test splits, and to widen its training data with the PKU-Market-PCB benchmark. After that, we plan to test the lighter FasterNet and attention-based neck, the Wise-IoU loss, true Grad-CAM++ hooks, and the multi-view inference mode already present in the engine. Each change will be judged under the same audited protocol, so that any gain can be attributed to the change itself.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology Kongu Engineering College, Erode, India for the laboratory computing resources and technical guidance provided during this research.

REFERENCES

[1] P. Bhattacharya and M. F. Cloutier, "Explainable artificial intelligence for industrial applications," in Proc. IEEE Int. Conf. Ind. Technol. (ICIT), 2022, pp. 1-8.

[2] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV), 2018, pp. 839-847.

[3] J. Chen, K. Cao, and Y. Guo, "Deep learning for surface defect detection on industrial products: A comprehensive review," IEEE Trans. Instrum. Meas., vol. 70, pp. 1-18, 2021.

[4] J. Chen, S. Kao, H. He, W. Zhuo, S. Wen, C. Lee, and S. Yu, "Run, don't walk: Chasing higher FLOPS for faster neural networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2023, pp. 12021-12031.

[5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," Int. J. Comput. Vis., vol. 88, no. 2, pp. 303-338, 2010.

[6] IPC-A-610G, Acceptability of Electronic Assemblies, IPC — Association Connecting Electronics Industries, Bannockburn, IL, USA, 2020.

[7] G. Jocher and J. Qiu, "Ultralytics YOLOv11: State-of-the-art real-time computer vision," Ultralytics Inc., 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] Q. Ling and N. A. M. Isa, "Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey," IEEE Access, vol. 11, pp. 15921-15944, 2023.

[9] D. Ouyang, S. He, G. Zhang, M. Luo, H. Guo, J. Zhan, and Z. Huang, "Efficient multi-scale attention module with cross-spatial learning," in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), 2023, pp. 1-5.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 779-788.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vis. (ICCV), 2017, pp. 618-626.

[12] M. Shakya et al., "FICS-PCB: A multi-modal image dataset for automated printed circuit board visual inspection," IACR Cryptol. ePrint Arch., vol. 2020, p. 1422, 2020.

[13] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling methods for object detection measures," Image Vis. Comput., vol. 107, p. 104117, 2021.

[14] S. Tang, F. He, X. Huang, and J. Yang, "Online PCB defect detector on a new PCB defect dataset," arXiv preprint arXiv:1902.06197, 2019.

[15] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," IEEE Trans. Ind. Informat., vol. 15, no. 4, pp. 2405-2415, 2019.

[16] Z. Tong, Y. Chen, J. Xu, and R. Yu, "Wise-IoU: Bounding box regression loss with dynamic focusing mechanism," arXiv preprint arXiv:2301.10051, 2023.

[17] X. Ye, L. Huang, J. Wang, X. Hu, J. Xu, and C. Zhang, "A lightweight PCB defect detection algorithm based on an improved YOLOv11s," IEEE Access, vol. 14, pp. 56412-56426, 2026, doi: 10.1109/ACCESS.2026.3678967.

[18] Z. Zhang, "Improved Adam optimizer for deep neural network training in industrial visual inspection," IEEE Trans. Ind. Electron., vol. 68, no. 8, pp. 7412-7422, 2021.